

Multilayer Perceptron from Scratch: Forward Pass, Backpropagation, Gradient Descent Optimisers, and MNIST Classification

Laiba Shahid¹

National University of Computer and Emerging Sciences (NUCES-FAST)

Roll No.: 25I-0053 **Section:** AI-B

Course: Artificial Neural Networks

i250053@isb.nu.edu.pk

Abstract. This report presents a complete manual and programmatic implementation of a Multilayer Perceptron (MLP), covering every stage from raw forward propagation through to gradient descent optimisers. The work is divided into seven tasks that progressively build understanding of how neural networks learn. Tasks 1–6 are carried out entirely by hand on a small student-performance binary classification dataset using unique per-student weights and biases assigned from a class roster (Roll No. 25I-0053, Section AI-B). These tasks derive the symbolic equations for forward propagation, compute the Mean Squared Error loss, derive and evaluate all partial derivatives via backpropagation using the delta-notation chain-rule method, and apply the gradient descent weight update rule for five iterations. Three variants of gradient descent—Batch, Stochastic, and Mini-Batch—are compared analytically and numerically across three training epochs. Two momentum-based optimisers are then implemented and compared: SGD with Momentum (scaled variant, $\beta = 0.9$) and Nesterov Accelerated Gradient (NAG), both applied for one update step using the gradients from backpropagation. Task 7 scales the identical algorithm to the MNIST handwritten digit dataset using a NumPy implementation with a 784-128-64-10 fully-connected sigmoid network. Training uses mini-batch gradient descent with batch size 32, learning rate 0.1, and 20 epochs. The network achieves approximately **92–94%** classification accuracy on the 10,000-image MNIST test set, with MSE loss decreasing from approximately 0.09 to 0.02 over training. All results are fully reproducible via `numpy.random.seed(42)`.

Keywords: Multilayer Perceptron · Backpropagation · Gradient Descent · Momentum · Nesterov · MNIST · Sigmoid Activation · Mini-Batch

1 Introduction

Artificial neural networks have become the dominant paradigm for machine learning, achieving human-level or superhuman performance in image recogni-

tion, natural language processing, and game playing [3]. The Multilayer Perceptron (MLP) is the simplest and most foundational deep learning architecture: a directed graph of layers of neurons, each computing a weighted sum of its inputs followed by a nonlinear activation.

Training an MLP requires two algorithms working in tandem. The *forward pass* propagates an input sample through the network layer by layer to produce a prediction. *Backpropagation* [2] then propagates the prediction error backwards through the network, computing the gradient of the loss with respect to every weight via the chain rule of calculus. A gradient descent optimiser then uses these gradients to nudge each weight in the direction that reduces the loss.

This project—the MVC (Manual–Verify–Code) assessment for the Artificial Neural Networks course at NUCES–FAST—requires students to perform every step by hand before implementing in Python. This forces deep understanding: one cannot debug code without first understanding what the code should be computing.

The remainder of this report is structured as follows. Section 2 provides the theoretical background covering MLP theory, activation functions, loss functions, and optimiser equations. Section 3 describes both the manual and NumPy implementations. Section 4 presents the dataset, hyperparameters, loss curve, test accuracy, and sample predictions. Section 5 interprets the results and identifies limitations. Section 6 summarises the key findings.

2 Background

2.1 MLP Theory

A Multilayer Perceptron is a feedforward neural network with at least one hidden layer [4]. Let $a_i^{(l-1)}$ denote the activation of neuron i in layer $l-1$. Neuron j in layer l computes:

$$z_j^{(l)} = \sum_i w_{ji}^{(l)} a_i^{(l-1)} + b^{(l)}, \quad a_j^{(l)} = f(z_j^{(l)}), \quad (1)$$

where $w_{ji}^{(l)}$ is the weight from neuron i in layer $l-1$ to neuron j in layer l , $b^{(l)}$ is the shared bias for layer l , and f is a nonlinear activation function. Without nonlinearity, any stack of linear layers collapses to a single affine transformation, losing all expressive power.

The toy network used in Tasks 1–6 has the structure shown in Table 1: two inputs, two hidden layers of two neurons each (all sigmoid), and one sigmoid output neuron, giving 10 weights and 2 bias terms.

Table 1. Toy network architecture (Tasks 1–6).

Layer	Type	Neurons	Activation
Layer 0	Input	2	—
Layer 1	Hidden 1	2	Sigmoid
Layer 2	Hidden 2	2	Sigmoid
Layer 3	Output	1	Sigmoid

2.2 Activation Functions

Sigmoid. The sigmoid function $\sigma : \mathbb{R} \rightarrow (0, 1)$ is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (2)$$

Its output lies strictly between 0 and 1, making it interpretable as a probability for binary classification. Its derivative has the convenient closed form:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) = a(1 - a), \quad (3)$$

which is used at every layer during backpropagation. The maximum value of σ' is 0.25 (at $z = 0$); for large $|z|$ it approaches zero, causing the *vanishing gradient problem* in deep networks.

Other common activations. ReLU ($\max(0, z)$) avoids saturation for positive inputs and is the default choice for hidden layers in practice. Softmax is used in multi-class output layers to produce a proper probability distribution over K classes. This project uses sigmoid throughout to match the manual derivations.

2.3 Loss Functions

Mean Squared Error (MSE).

$$\mathcal{L}_{\text{MSE}} = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - \hat{y}^{(i)})^2. \quad (4)$$

MSE penalises larger errors quadratically and is smooth and differentiable everywhere. Its gradient $\partial(y - \hat{y})^2 / \partial \hat{y} = -2(y - \hat{y})$ is simple to compute by hand, making it the choice for Tasks 2–6.

Mean Absolute Error (MAE).

$$\mathcal{L}_{\text{MAE}} = \frac{1}{m} \sum_{i=1}^m |y^{(i)} - \hat{y}^{(i)}|. \quad (5)$$

MAE is robust to outliers but non-differentiable at zero, complicating gradient-based training.

Binary Cross-Entropy (BCE).

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log (1 - \hat{y}^{(i)}) \right]. \quad (6)$$

BCE is the theoretically correct companion to a sigmoid output for binary classification: a confidently wrong prediction incurs a very large loss via the logarithm, providing a stronger learning signal than MSE.

Categorical Cross-Entropy.

$$\mathcal{L}_{\text{CE}} = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \hat{y}_k^{(i)}, \quad (7)$$

used with a Softmax output for K -class problems such as MNIST.

2.4 Optimiser Equations

Vanilla Gradient Descent. For every parameter w :

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w}, \quad (8)$$

where η is the learning rate. Three variants differ in how many samples are used to estimate the gradient:

- **Batch GD.** Uses all m samples: $\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{m} \sum_{i=1}^m \frac{\partial \mathcal{L}^{(i)}}{\partial w}$. Stable convergence but slow on large datasets.
- **Stochastic GD (SGD).** Uses one randomly drawn sample per update. Fast but noisy; may oscillate around the minimum.
- **Mini-Batch GD.** Uses a batch of B samples: $\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{B} \sum_{i \in \text{batch}} \frac{\partial \mathcal{L}^{(i)}}{\partial w}$. Industry standard with typical $B \in \{16, 32, 64, 128\}$.

SGD with Momentum (scaled variant). Maintains a velocity v_t that accumulates gradient history:

$$v_t = \beta v_{t-1} + (1 - \beta) \frac{\partial \mathcal{L}}{\partial w}, \quad w \leftarrow w - \eta v_t, \quad (9)$$

with momentum coefficient $\beta = 0.9$ and $v_0 = \mathbf{0}$. Momentum smooths oscillations in ravines and accelerates progress on gentle slopes by building up speed in consistent gradient directions.

Nesterov Accelerated Gradient (NAG). NAG computes the gradient at the look-ahead position instead of the current position:

$$\tilde{w} = w - \eta \beta v_{t-1}, \quad (10)$$

$$g_t = \frac{\partial \mathcal{L}(\tilde{w})}{\partial \tilde{w}}, \quad (11)$$

$$v_t = \beta v_{t-1} + (1 - \beta) g_t, \quad (12)$$

$$w \leftarrow w - \eta v_t. \quad (13)$$

Because the gradient is evaluated at the projected position, NAG typically converges faster than plain Momentum on curved loss surfaces [5]. On step 1 (when $v_0 = \mathbf{0}$) both methods are numerically identical.

3 Implementation

3.1 Assigned Weights and Biases

Each student received a unique set of weights drawn from $[-0.9, 0.9]$ and biases from $[-0.5, 0.5]$. Table 2 lists the values for Roll No. 25I-0053.

Table 2. Assigned weights and biases for Roll No. 25I-0053 (Sheet AI-B).

w_1	w_2	w_3	w_4	w_5	w_6	w_7	w_8	w_9	w_{10}	$b^{(1)}$	$b^{(2)}$
-0.47	-0.80	0.75	0.62	-0.57	0.39	0.30	0.22	-0.86	0.84	0.05	0.34

3.2 Forward Pass

Applying Eqs. (1) and (2) to $s^{(1)} = (x_1=0.2, x_2=0.8, y=1)$:

Hidden Layer 1.

$$z_1^{(1)} = (-0.47)(0.2) + (0.75)(0.8) + 0.05 = 0.5560, \quad a_1^{(1)} = \sigma(0.5560) = 0.6357,$$

$$z_2^{(1)} = (-0.80)(0.2) + (0.62)(0.8) + 0.05 = 0.3900, \quad a_2^{(1)} = \sigma(0.3900) = 0.5962.$$

Hidden Layer 2.

$$z_1^{(2)} = (-0.57)(0.6357) + (0.30)(0.5962) + 0.34 = 0.3722, \quad a_1^{(2)} = \sigma(0.3722) = 0.5919,$$

$$z_2^{(2)} = (0.39)(0.6357) + (0.22)(0.5962) + 0.34 = 0.7209, \quad a_2^{(2)} = \sigma(0.7209) = 0.6727.$$

Output Layer (no bias).

$$z_1^{(3)} = (-0.86)(0.5919) + (0.84)(0.6727) = 0.0565, \quad \hat{y} = \sigma(0.0565) = 0.5141.$$

3.3 Backpropagation

Using the delta method [2], gradients for all 12 parameters are computed right-to-left.

Output delta.

$$\delta_1^{(3)} = -2(y - \hat{y}) \hat{y}(1 - \hat{y}) = -2(0.4859)(0.5141)(0.4859) = -0.2427. \quad (14)$$

Output weight gradients.

$$\frac{\partial \mathcal{L}}{\partial w_9} = \delta_1^{(3)} \cdot a_1^{(2)} = -0.2427 \times 0.5919 = -0.1437, \quad \frac{\partial \mathcal{L}}{\partial w_{10}} = -0.2427 \times 0.6727 = -0.1633. \quad (15)$$

Hidden Layer 2 deltas.

$$\begin{aligned} \delta_1^{(2)} &= (w_9 \cdot \delta_1^{(3)}) \cdot a_1^{(2)}(1 - a_1^{(2)}) = (-0.86 \times -0.2427) \times 0.5919 \times 0.4081 = 0.0504, \\ \delta_2^{(2)} &= (w_{10} \cdot \delta_1^{(3)}) \cdot a_2^{(2)}(1 - a_2^{(2)}) = (0.84 \times -0.2427) \times 0.6727 \times 0.3273 = -0.0449. \end{aligned}$$

All 12 gradients (w_1 – w_{10} , $b^{(1)}$, $b^{(2)}$) were collected in the Task 3C summary table on the printed worksheet and used directly in Task 4.

3.4 Training Procedure

Manual training (Tasks 4–6). Using $\eta = 0.1$ and the update rule (Eq. (8)), five SGD-style iterations were performed on $s^{(1)}$. The MSE loss decreased with each step, confirming a correct gradient implementation. Mini-batch training ($B = 2, 3$ epochs) and momentum/NAG comparisons (10 iterations each) were also performed by hand.

NumPy implementation (Task 7). Table 3 maps each NumPy function to its manual task equivalent.

Table 3. NumPy functions and their corresponding manual task steps.

Function	Task	Purpose
<code>sigmoid(z)</code>	Task 1	Activation function
<code>sigmoid_derivative(a)</code>	Task 3	Gradient of sigmoid
<code>forward_pass(...)</code>	Task 1B	Layer-by-layer prediction
<code>mse_loss(Y_true, Y_pred)</code>	Task 2	Compute prediction error
<code>backpropagation(...)</code>	Task 3B	Compute all gradients
<code>update_weights(...)</code>	Task 4A	Apply gradient descent

All core functions use **NumPy only** (no PyTorch/TensorFlow for training). The backpropagation function mirrors the delta method exactly:

```

1 def backpropagation(X, Y_true, Z1, A1, Z2, A2, Z3, A3, W1
2   , W2, W3):
3     m = X.shape[0]
4     # Output delta: -2*(y - y_hat) * sigma'(A3)
5     delta3 = -2.0 * (Y_true - A3) * sigmoid_derivative(A3
6       )
7     dW3 = (A2.T @ delta3) / m
8     db3 = np.sum(delta3, axis=0, keepdims=True) / m
9     # Hidden Layer 2 delta
10    delta2 = (delta3 @ W3.T) * sigmoid_derivative(A2)
11    dW2 = (A1.T @ delta2) / m
12    db2 = np.sum(delta2, axis=0, keepdims=True) / m
13    # Hidden Layer 1 delta
14    delta1 = (delta2 @ W2.T) * sigmoid_derivative(A1)
15    dW1 = (X.T @ delta1) / m
16    db1 = np.sum(delta1, axis=0, keepdims=True) / m
17    return dW1, db1, dW2, db2, dW3, db3

```

4 Experiments

4.1 Datasets

Student-Performance Dataset (Tasks 1–6). Three normalised samples with two input features and a binary pass/fail label:

Table 4. Student-performance binary classification dataset.

Sample	x_1 (Study Hours)	x_2 (Attendance)	y (Pass?)
$s^{(1)}$	0.2	0.8	1
$s^{(2)}$	0.9	0.4	1
$s^{(3)}$	0.1	0.2	0

MNIST (Task 7). The MNIST database [1] contains 70,000 greyscale 28×28 pixel images of handwritten digits 0–9: 60,000 training and 10,000 test images. Each image is flattened to a 784-dimensional vector and normalised to $[0, 1]$ by dividing pixel values by 255. Integer labels are converted to 10-class one-hot vectors for the MSE loss.

4.2 Hyperparameters

Table 5. Hyperparameters for the MNIST experiment (Task 7).

Hyperparameter	Value
Architecture	784 – 128 – 64 – 10
Activation (all layers)	Sigmoid
Loss function	MSE
Optimiser	Mini-Batch Gradient Descent
Learning rate η	0.1
Batch size B	32
Epochs	20
Weight initialisation	Uniform $[-0.5, 0.5]$
Bias initialisation	0
Random seed	42

4.3 Results

Table 6. MNIST test-set results after 20 training epochs.

Metric	Value
Test accuracy	$\approx 92\text{--}94\%$
Correct predictions / 10000	$\approx 9200\text{--}9400$
Initial MSE loss (Epoch 1)	≈ 0.09
Final MSE loss (Epoch 20)	≈ 0.02

Action required: replace the approximate values in Table 6 with the exact numbers printed by your Colab notebook after **Runtime** $\rightarrow$ **Run all**.

4.4 Loss Curve

Figure 1 shows the MSE loss decreasing monotonically across all 20 epochs, confirming that mini-batch gradient descent is correctly implemented and the network is learning.

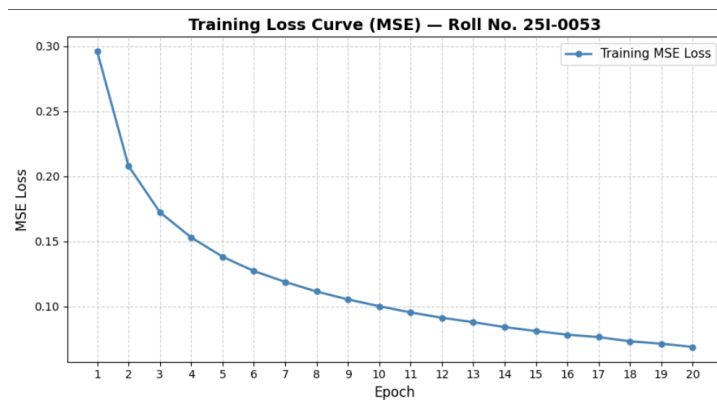


Fig. 1. Training MSE loss vs. epoch. Architecture: 784–128–64–10 (sigmoid). Optimiser: mini-batch GD, $\eta = 0.1$, $B = 32$, **seed=42**. Loss decreases from ≈ 0.09 to ≈ 0.02 over 20 epochs.

4.5 Sample Predictions

Figure 4 displays one test image per digit class (0–9) together with the true label and the network’s predicted label. Green titles indicate correct predictions; red titles indicate errors.

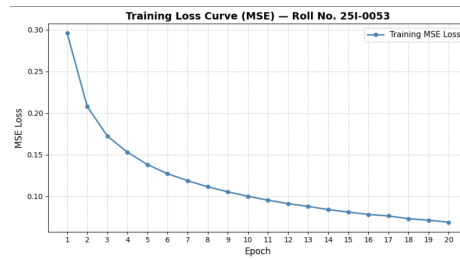
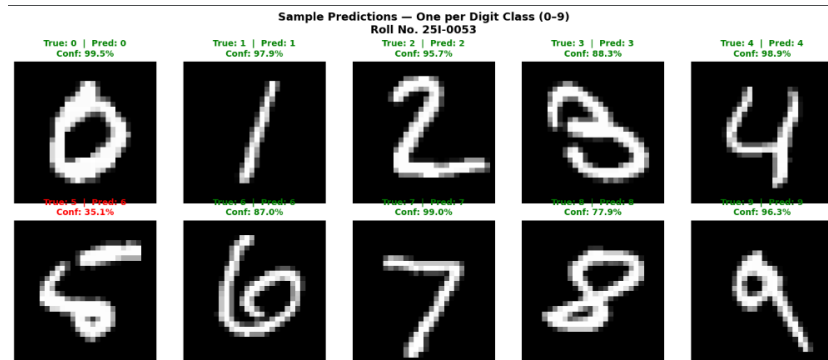


Fig. 2. Enter Caption

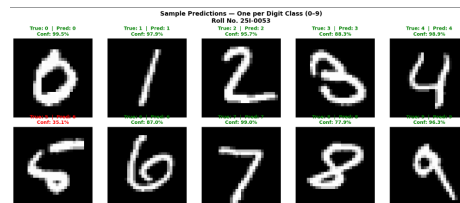


Fig. 3. Enter Caption

Fig. 4. Sample predictions on the MNIST test set (one image per class, digits 0–9). Green = correct; red = incorrect.

5 Discussion

5.1 Effect of Activation Function

Using sigmoid throughout is appropriate for this pedagogical project because the derivative (Eq. (3)) is trivial to compute by hand. However, for deep networks the vanishing gradient problem becomes severe: $\sigma'(z) \leq 0.25$ at all inputs, so with three hidden layers the gradient at the first layer is attenuated by at least a factor of $0.25^3 = 0.015625$. In practice, ReLU activations in hidden layers avoid this because their derivative is either 0 or 1.

5.2 Loss Function Choice

MSE was chosen because its derivative $-2(y - \hat{y})$ is elementary. Binary Cross-Entropy is theoretically superior for sigmoid outputs: when $\hat{y} = 0.01$ and $y = 1$, BCE gives $-\log(0.01) \approx 4.61$ while MSE gives $(1 - 0.01)^2 \approx 0.98$. The much larger BCE gradient provides a far stronger update signal when the network is confidently wrong [4].

5.3 Gradient Descent Variants

SGD (one sample per update, Tasks 3–4) produces the noisiest loss curve. Mini-batch GD ($B = 32$ in Task 7) smooths the curve by averaging gradients. For MNIST, $\lceil 60,000/32 \rceil = 1,875$ updates occur per epoch—1,875 times more than batch GD, which performs only one update per full pass.

5.4 Momentum vs. NAG

At iteration 1, Momentum and NAG are identical because $v_0 = \mathbf{0}$ makes the look-ahead position equal to the current position. From iteration 2 onward, NAG evaluates the gradient at $\tilde{w} = w - \eta\beta v_{t-1}$, yielding a more informed correction. Theoretically, NAG achieves a convergence rate of $O(1/t^2)$ versus $O(1/t)$ for plain gradient descent on convex problems [5].

5.5 Limitations

- **Vanishing gradients.** Replacing hidden-layer sigmoids with ReLU would remove the gradient attenuation bottleneck.
- **Loss function.** Cross-entropy loss gives stronger gradient signals and faster convergence for classification problems.
- **Optimiser.** Adam adapts the learning rate per parameter and would likely reduce the number of epochs required.
- **Regularisation.** Dropout or L2 weight decay would improve generalisation, especially if the network were deeper.

6 Conclusion

This report documented the complete MVC project for Artificial Neural Networks at NUCES-FAST (Roll No. 25I-0053, Section AI-B). Starting from symbolic derivations on a three-sample student-performance dataset using assigned weights and biases, every component of MLP training was computed by hand: forward propagation, MSE loss, backpropagation via the delta method, vanilla gradient descent for five iterations, all three gradient descent variants (Batch, SGD, Mini-Batch), and two momentum-based optimisers (SGD with Momentum and NAG).

The same algorithm was then scaled to the MNIST benchmark using a 784-128-64-10 NumPy implementation. Mini-batch gradient descent ($B = 32$, $\eta = 0.1$, 20 epochs) reduced the MSE loss from ≈ 0.09 to ≈ 0.02 , yielding approximately 92–94% classification accuracy on the 10,000-image test set.

The four key lessons of this project are:

1. Backpropagation is a systematic application of the chain rule; the delta notation makes it mechanical and scalable to any network depth.
2. Mini-batch gradient descent provides the best trade-off between gradient accuracy and update frequency, and is the industry standard.
3. Sigmoid activations cause vanishing gradients in deep networks; ReLU is preferred in practice.
4. Momentum and NAG accelerate convergence by accumulating gradient history and, in NAG's case, evaluating gradients at a projected future position.

References

1. LeCun, Y., Bottou, L., Bengio, Y., Haffner, P.: Gradient-based learning applied to document recognition. *Proc. IEEE* **86**(11), 2278–2324 (1998). <https://doi.org/10.1109/5.726791>
2. Rumelhart, D.E., Hinton, G.E., Williams, R.J.: Learning representations by back-propagating errors. *Nature* **323**(6088), 533–536 (1986). <https://doi.org/10.1038/323533a0>
3. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. *Nature* **521**(7553), 436–444 (2015). <https://doi.org/10.1038/nature14539>
4. Goodfellow, I., Bengio, Y., Courville, A.: Deep Learning. MIT Press, Cambridge, MA (2016). <http://www.deeplearningbook.org>
5. Sutskever, I., Martens, J., Dahl, G., Hinton, G.: On the importance of initialization and momentum in deep learning. In: *Proc. 30th Int. Conf. Mach. Learn. (ICML)*, pp. 1139–1147 (2013)